

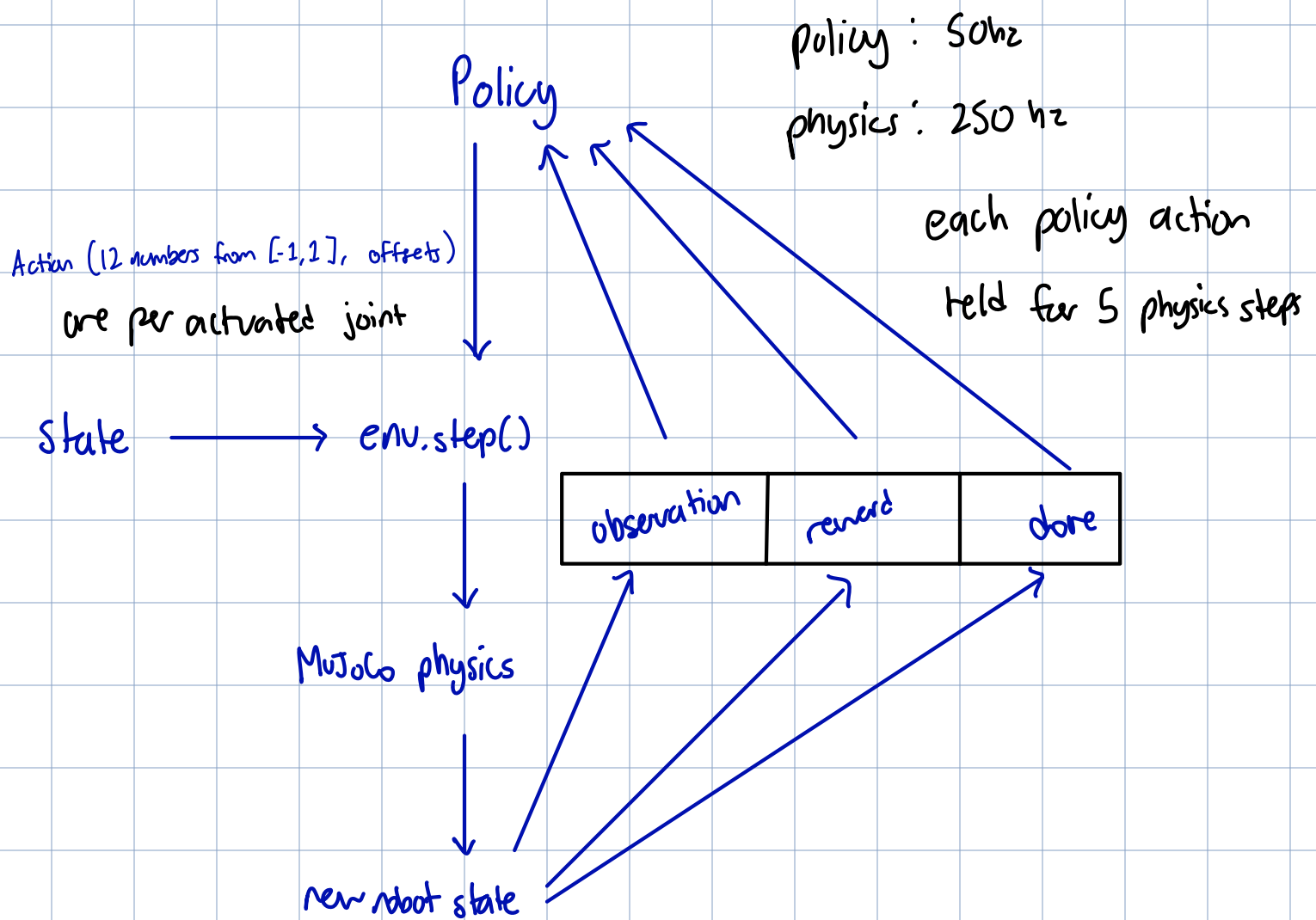
Stage 3 : The MJX Environment

RL environment defines the contract

$(\text{state}, \text{action}) \longrightarrow (\text{next state}, \text{observation}, \text{reward})$

Brax vectorizes this environment over thousands of robots

robot command consists of forward speed, lateral speed, yaw rate



Part 1: `- get_obs(data, info) → jnp.array[45]`

observation consists of 45 numbers

0	3	6	9	21	33	45
gyno	gravity	command	joint position offsets	joint velocity	last action	
3	3	3	12	12	12	

don't want to train on unrealistic perfect sensor data,
so add small random noise

Part 2: `- get_termination(data) → bool`

termination conditions:

trunk up vector $z < 0$ (flipped over)

trunk height $q_{pos}[2] < 0.12m$

any element of q_{pos} is not finite (NaN messes policy up)

if any of these are true, terminate run

Part 3 : `sample_command(rng) → jnp.ndarray[3]`

decides what movement pup should attempt during training



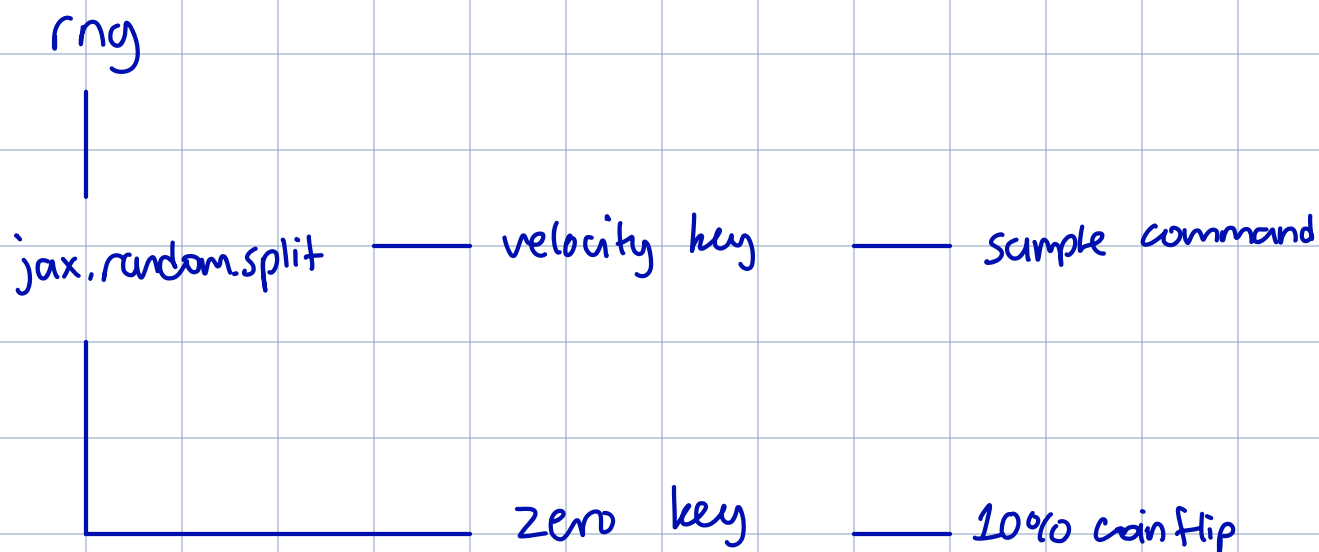
split rng (don't use same key for multiple draws in JAX)



randomly choose vx, vy, wz



make 10% zero command



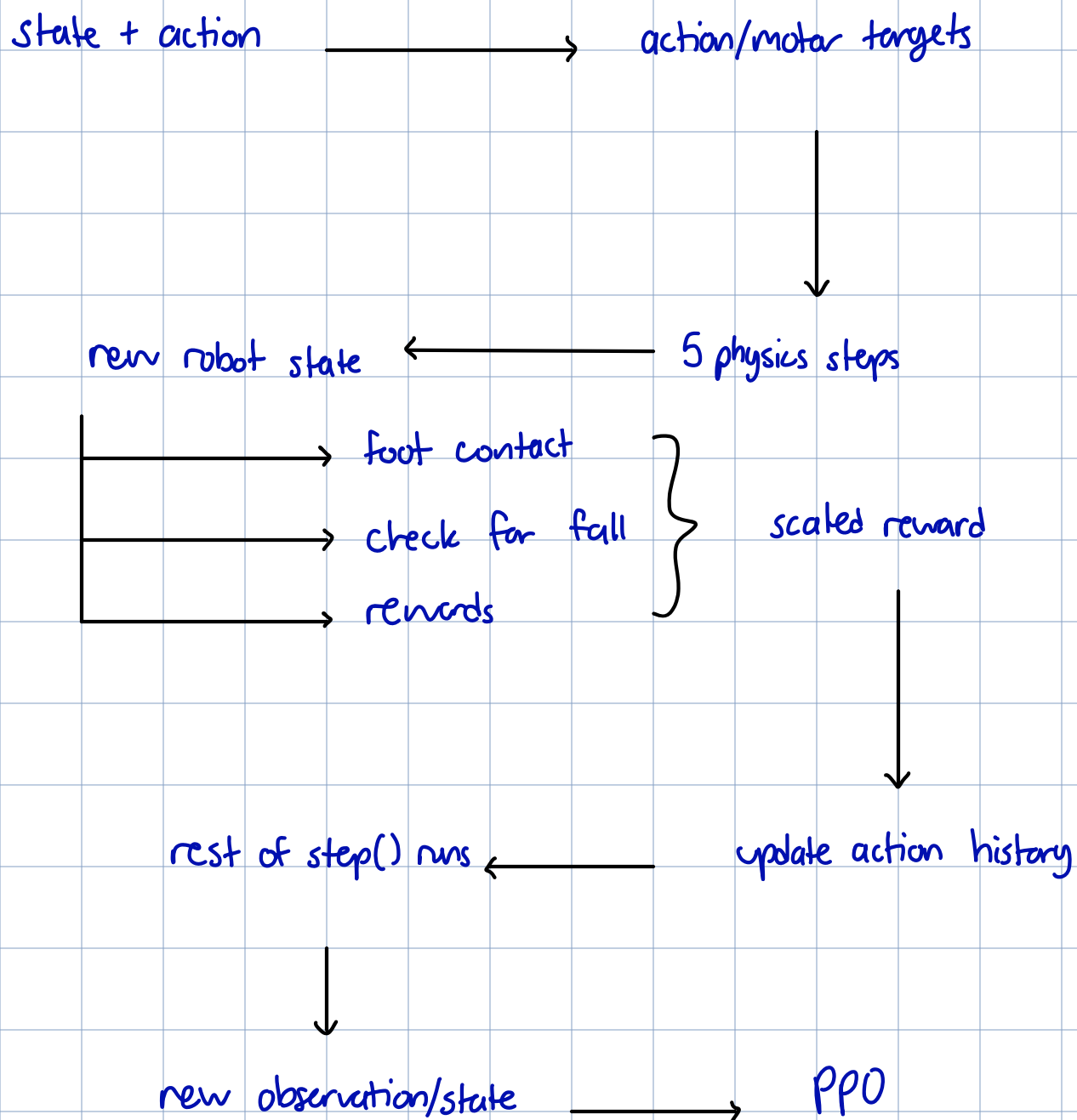
`jnp.where()` at end to avoid branchi

Part 4: the core of step (state, action)

takes policy actions as input

moves pup forward by 0.02 seconds

determines what happened, calculates reward, returns new state



Part 5: 3 reward terms in rewards.py

rewards: Moving at the commanded linear velocity?

Turning at the commanded angular velocity?

higher reward $\longrightarrow$ higher

small sigma $\longrightarrow$ error hurts reward quickly

large sigma $\longrightarrow$ more forgiving

costs: Actions smooth or jerky?

higher cost $\longrightarrow$ worse

Policy actions $\longrightarrow$ pup moves

